Chapter V – Supervised Learning

Instituto Superior de Engenharia
Politécnico de Coimbra

Simão Paredes
sparedes@isec.pt

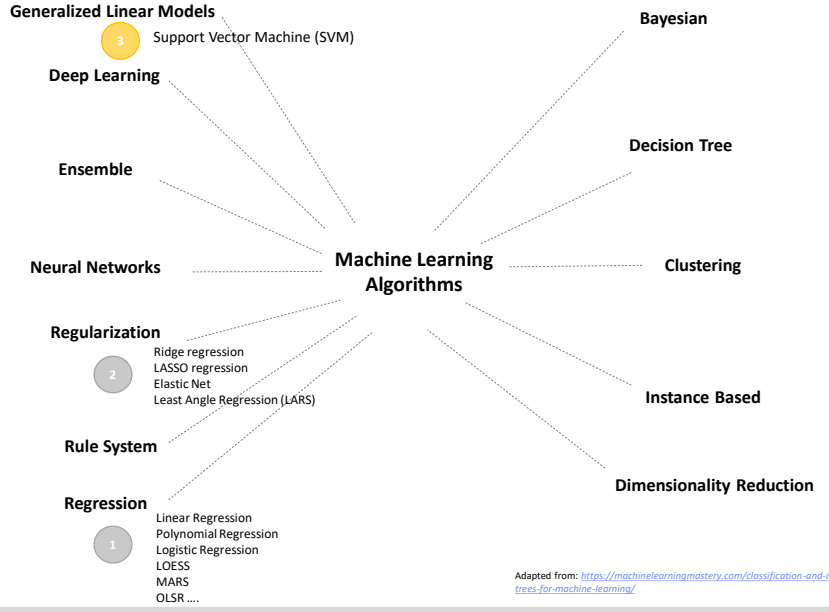
Classification

Supervised Learning Algorithms

Support Vector Machines (SVM)

Classification

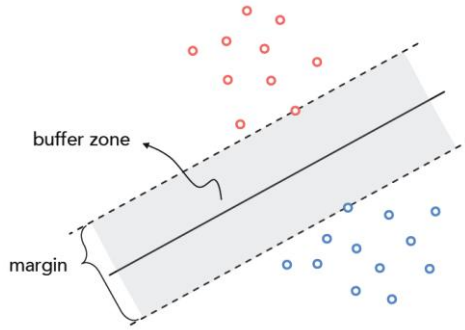
Machine Learning Algorithms



Adapted from: https://machinelearningmastery.com/classification-and-regression-trees-for-machine-learning/

Support Vector Machine (SVM)

- SVM Classifier:
  - based on a hyperplane in multidimensional space to separate different classes.
    - the hyperplane is created iteratively in order to minimize the error.
    - the main goal is to find the maximum marginal hyperplane (MMH)
      - i.e. the hyperplane that best divides the dataset into classes.

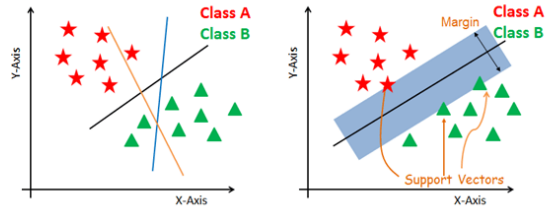


Support Vector Machine (SVM)

- SVM Classifier:
  - can handle continuous and categorical variables
  - is highly sensitive to differences in features' scale
    - Feature scaling is mandatory
  - is versatile can be applied to several problems:
    - Classification / Regression
      - SVM is often regarded exclusively as a classification approach, however it also can be applied to regression problems.
    - Particularly well adapted to classification of complex small or medium sized datasets

SVM: Linear Classification

- Linear Classification

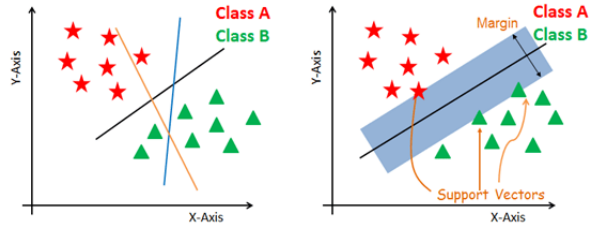


https://www.datacamp.com/community/tutorials/svm-classification-scikit-learn-python

- Hyperplane is a decision plane that separates instances with different classes.
  - The main objective is to identify **a hyperplane with the maximum possible margin between support vectors** (nearest instances from different classes) in a given dataset.

SVM: Linear Classification

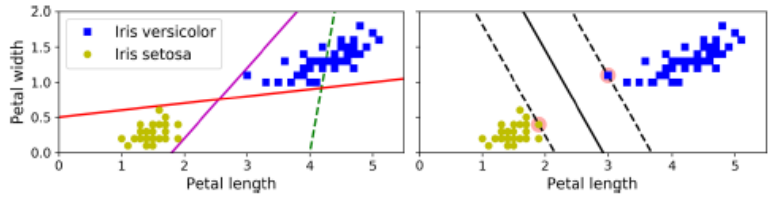
- Iterative process:
  - Generate hyperplanes which correctly discriminates the classes.
    - **orange e blue are not candidates** as they cannot properly separate the data (different classes)
    - **black hyperplane is a candidate** ,as it separates the two classes



  - The main goal is the selection of the right hyperplane, i.e. with the **maximum segregation** from the nearest instances.

SVM: Linear Classification

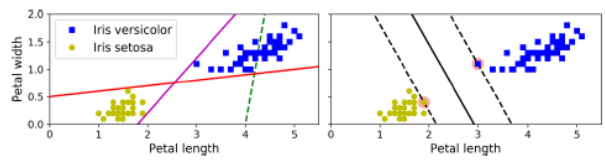
- Linear Classification



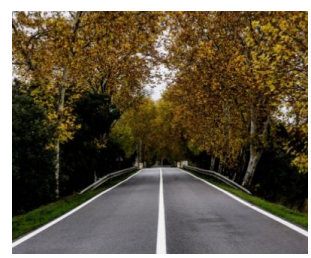
  - Model with the green dashed line?
    - Unacceptable!! It does not separate the classes
  - Models with solid lines (left image)?
    - Separate the classes **but** they are very close to the instances, probably this will originate a high rate of misclassification in unseen data.
  - Solid line (black) on the right image?
    - it represents the decision with highest margin (Largest Margin Classification)

SVM: Linear Classification

- Largest Margin Classification



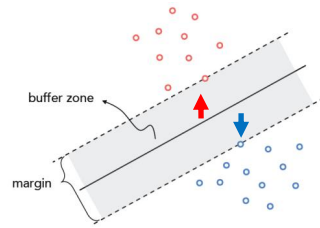
Not only separates the classes but also stays as far as possible from the closest training instances



Geron A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems 3rd Edition, ISBN-13: 978-1492032649, O'Reilly 2022

SVM: Linear Classification

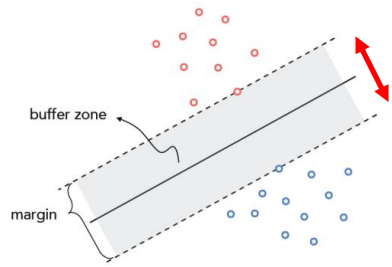
- Support Vectors
  - are the closest instances to the optimal hyperplane.
  - define the margin and, consequently, the position and orientation of the optimal hyperplane.



  - These instances are very relevant to the construction of the classifier.
    - If you remove any non support vector data points, the hyperplane would not change.
    - This makes SVMs very efficient as only a subset of the training data is used in the final model definition.

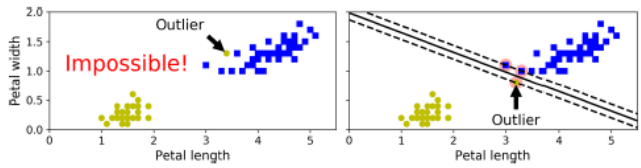
SVM: Linear Classification

- Margin
  - is a gap between the two support vectors (closest different class instances).
  - is calculated as the perpendicular distance from the hyperplane to support vectors.
    - **Higher the margin** the **better** is the SVM classifier.



SVM: Linear Classification

- Hard Margin vs Soft Margin Classification
  - Hard Margin
    - Impose that **all the instances must be off the margin area and on the right side**
    - Two main issues:
      - It only works if the data is linearly separable
      - It is very sensitive to outliers

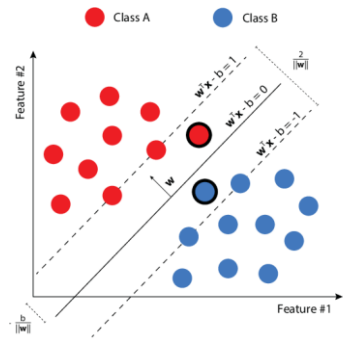


a)
In this case the presence of **1 outlier** blocks the classification problem, i.e. it is not possible to find an hyperplane that can divide the two classes

b)
1 outlier is imposing a significant reduction in the classification margin (lower the margin the lower is the classifier performance

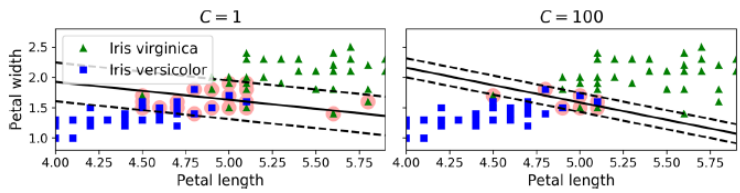
SVM: Linear Classification

- Soft Margin Classification
  - seeks for a balance between the dimension of the margin and margin violations (instances in the margin area or in the wrong side of the decision hyperplane)
    - The level of allowed margin violation is an **hyperparameter** of the algorithm
    - It allows additional flexibility avoiding the issues created by hard margin classification



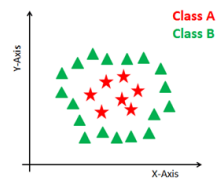
SVM: Linear Classification

- Hyperparameter C
  - is the soft margin regularization parameter and it controls how strongly the model penalizes margin violations.
    - Low value means wider margin and higher level of violations
    - High value, a low number of margin violations is allowed but implies a reduction in the margin
      - It may reduce the **classifier's generalization**



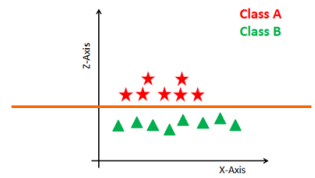
SVM: Nonlinear Classification

- Nonlinear Classification
  - There are some problems that cannot be solved using linear hyperplane.



https://www.datacamp.com/community/tutorials/svm-classification-scikit-learn-python

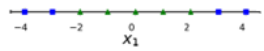
  - How to solve this?
    - SVM applies a data transformation (polynomial or based on a kernel) to transform the input space to a higher dimensional space where a linear separability can be applied



SVM: Nonlinear Classification

- Nonlinear dataset

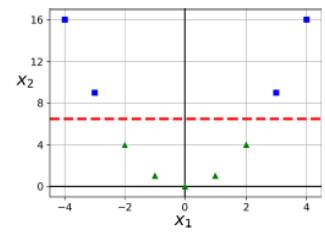
Impossible to implement a linear separation between the two classes



Data Transform
(create a new dimensional space, in this case a 2D dataset (x_1, x_2) based on x_1)

$$x_2 = (x_1)^2$$

In the new dataspace the data are linearly separable



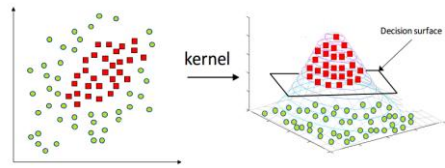
Polynomial features are simple to implement and can originate good results

Important Drawback:

Increase complexity, explosion of features!!

SVM: Nonlinear Classification

- Kernel Trick
  - It takes a low-dimensional input space and transforms it into a higher dimensional space:
    - Converting a nonseparable problem to a separable problems by adding more dimensions to it.



https://medium.com/@zxr.nju/what-is-the-kernel-trick-why-is-it-important-98a98db0961d

  - There are several kernels:
    - Linear Kernel
    - Polynomial Kernel
    - Gaussian Kernel
      - SVC class

```python
# Polynomial Kernel
from sklearn.svm import SVC
poly_kernel_svm_clf=Pipeline([
    ('scaler', StandardScaler()),
    ('svm-clf',SVC(kernel='poly',degree=3, coef0=1, C=5))

])
poly_kernel_svm_clf.fit(X,y)
```

SVM

- Decision Function / Predictions
  - Unlike linear regression where $\boldsymbol{\theta}$ denotes the weights vector, the notation for weights and bias in the SVM context is different:

$$\mathbf{w}, b$$

  - The linear SVM classifier predicts the class of a new instance $\mathbf{x}$ based on the result of:

$$\mathbf{w}^{\mathrm{T}}\mathbf{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

  - where the decision boundary is given by:

$$\mathbf{w}^{\mathrm{T}}\mathbf{x} + b = 0$$

SVM

- The linear SVM classifier model predicts the class $\hat{y}$ of a new instance $\mathbf{x}$ by simply computing:

$$\mathbf{w}^{\mathrm{T}}\mathbf{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

  - The output of the model is:

$$\hat{y} = \begin{cases} 1 & if \quad \mathbf{w}^{\mathrm{T}}\mathbf{x} + b > 0 \\ -1 & if \quad \mathbf{w}^{\mathrm{T}}\mathbf{x} + b < 0 \end{cases}$$

SVM

- Decision boundary:

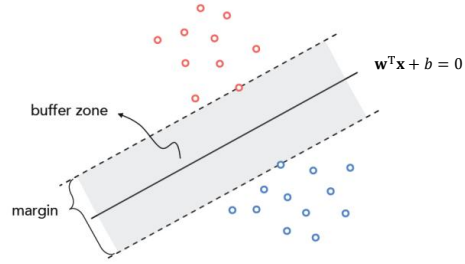
$$\mathbf{w}^{\mathrm{T}}\mathbf{x} + b = 0$$

  - Any instance above the decision boundary should have label 1
  - Any instance below the decision boundary should have label -1

$$\hat{y} = \begin{cases} 1 & if \quad \mathbf{w}^{\mathrm{T}}\mathbf{x} + b > 0 \\ -1 & if \quad \mathbf{w}^{\mathrm{T}}\mathbf{x} + b < 0 \end{cases}$$

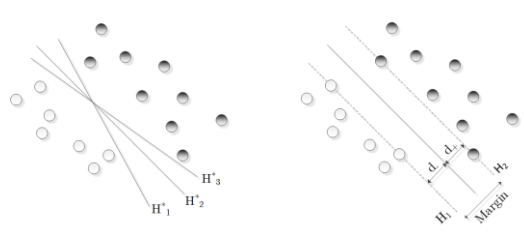
    - The labelling (-1, 1) allows a more condensed formulation for the decision function

$$\hat{y} = sign(\mathbf{w}^{\mathrm{T}}\mathbf{x} + b)$$



SVM

- It is possible to define several hyperplanes to separate the same training instances



  - The best hyperplane (optimal hyperplane) must be chosen such that:
    - **A small shift in the data should not result in a different class (predicted class)**
    - The higher the distance between the hyperplane and the nearest training instances the higher the generalization ability of the classification algorithm.

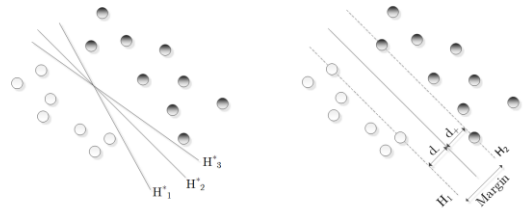
SVM

- The hyperplanes H must be defined:

$$H_1: \quad \mathbf{w}^T\mathbf{x} + \mathrm{b} = -\mathbf{1} \qquad \text{such that} \qquad \mathbf{w}^T\mathbf{x}^{(i)} + \mathrm{b} \leq -\mathbf{1} \quad \text{if} \quad y^{(i)} = -1$$

$$H_2: \quad \mathbf{w}^T\mathbf{x} + \mathrm{b} = \mathbf{1} \qquad\qquad\qquad\quad \mathbf{w}^T\mathbf{x}^{(i)} + \mathrm{b} \geq \mathbf{1} \quad \text{if} \quad y^{(i)} = +1$$

  - The margin is given by $(d_+ + d_-)$, where d_+ is the shortest distance to the closest positive point and d_- the shortest distance to the closest negative point

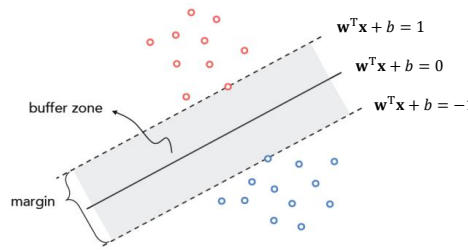


SVM

- According to the definition of hyperplanes:

$$\mathbf{w}^T\mathbf{x}^{(i)} + \mathrm{b} \leq -\mathbf{1} \quad \text{if} \quad y^{(i)} = -1$$

$$\mathbf{w}^T\mathbf{x}^{(i)} + \mathrm{b} \geq \mathbf{1} \quad \text{if} \quad y^{(i)} = +1$$



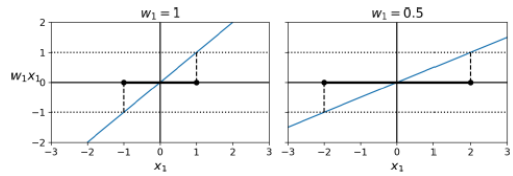
- Then, an instance i is properly classified by checking that:

$$y^{(i)}(\mathbf{w}^T\mathbf{x} + \mathrm{b}) \geq \mathbf{1}$$

  - **Training** a linear SVM classifier means finding the values of $\mathbf{w}$ and b that make the margin as wide as possible while avoiding margin violations (hard margin) or limiting them (soft margin)

SVM

- Considering the simplest case (one feature: $w_1 x_1;\ b = 0$):
  - The **smaller the weight vector, the larger the margin**



$$H_1: \quad \mathbf{w}^T\mathbf{x} + \mathrm{b} = -\mathbf{1};\ H_2: \quad \mathbf{w}^T\mathbf{x} + \mathrm{b} = \mathbf{1}$$

  - So the goal is to minimize the $\|\mathbf{w}\| = \sqrt{w_1^2 + \cdots + w_n^2}$ to get a larger margin.
    - The definition of the optimal hyperplanes is defined based on the nature of the classification process:
      - Hard Margin
      - Soft Margin

SVM

- Hard Margin optimization problem
  - Not allowed any margin violation
    - The decision function is greater than 1 for all positive training instances and lower than -1 for all negative instances
    - The hard margin linear SVM classifier objective can be formalized as:

$$minimize_{\mathbf{w},b} \quad \frac{1}{2}\mathbf{w}^T\mathbf{w}$$

$\frac{1}{2}\mathbf{w}^T\mathbf{w}$ has a simple derivative (**w**), which facilitates the optimization problem.

$$subject\ to \quad y^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right) \geq 1 \quad for\ \ i = 1,2,\ldots.,m$$

SVM

- Soft Margin optimization problem
  - $\xi^{(i)} \geq 0$ must be considered to define how much the ith instance is allowed to violate the margin
    - The value of $\xi^{(i)}$ should be as small as possible (low level of margin violation)

$$minimize_{\mathbf{w},b,\xi} \quad \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{m}\xi^{(i)}$$

$\xi^{(i)} = 0$ instance correctly classified outside the margin; $0 < \xi^{(i)} < 1$ correctly classified inside the margin; $\xi^{(i)} > 1$ misclassified

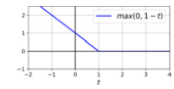
$$subject\ to \quad y^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right) \geq 1 - \xi^{(i)} \quad and \quad \xi^{(i)} \geq 0 \qquad for \quad i = 1,2,\ldots,m$$

  - The learning problem can also be formulated as an unconstrained optimization problem over $\mathbf{w}$

$$J(\mathbf{w}, b) = \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{m}\max(0,1 - y^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right))$$

1st term, small $\mathbf{w}$ means large margin
2nd term, controls margin violations

$\max(0,1-t)$ is the Hinge loss function



SVM: Algorithms

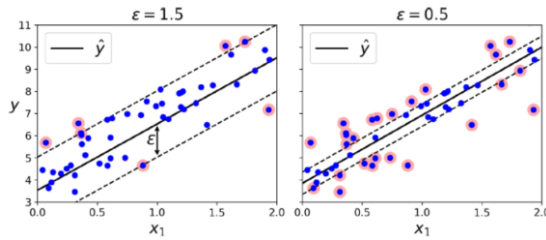
- Scikit Learn Algorithms for SVM Classification

Algorithm scikit-learn	Scaling Required	Kernel trick
LinearSVC	Yes	No
SGDClassifier	Yes	No
SVC	Yes	Yes

  - Remember!
    - SVM classifiers are very sensitive to scale differences among the several features

SVM

- SVM Regression
  - SVM algorithm also supports regression
    - However the **objective in regression is the opposite of the classification process**, instead of identifying the largest possible margin minimizing violations, here the goal is:
      - To fit as many instances as possible on the margin while limiting the margin violations (in this case the **instances outside the margin**)
        - The width of the margin is controlled by the hyperparameter ε



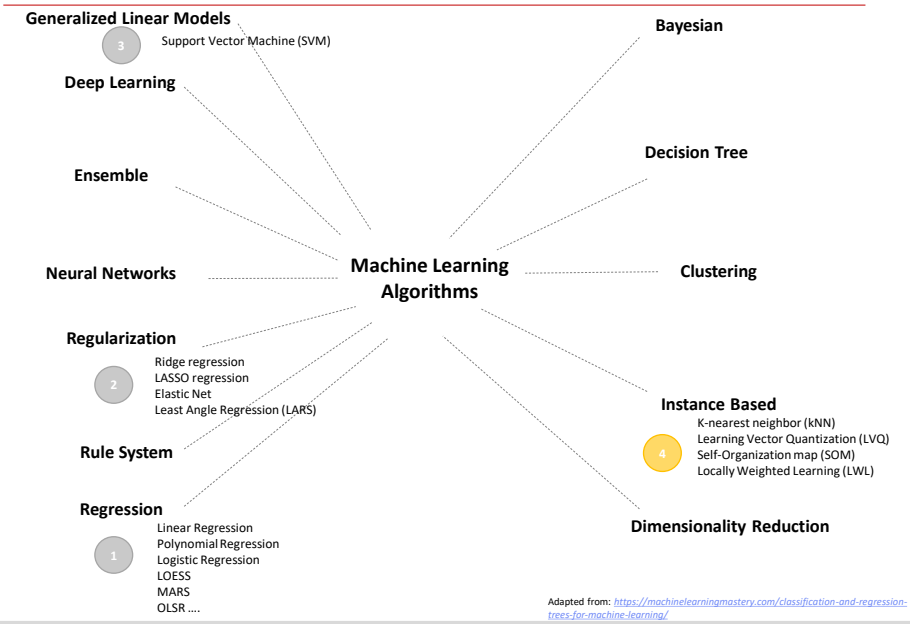
Adding more training instances inside the margin does not affect the model´s prediction ability

Geron A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems 2nd Edition, ISBN-13: 978-1492032649, O'Reilly 2019

Instance Based: k – Nearest Neighbor

Classification

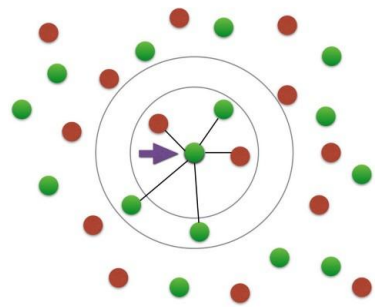
Machine Learning Algorithms



Adapted from: https://machinelearningmastery.com/classification-and-regression-trees-for-machine-learning/

kNN

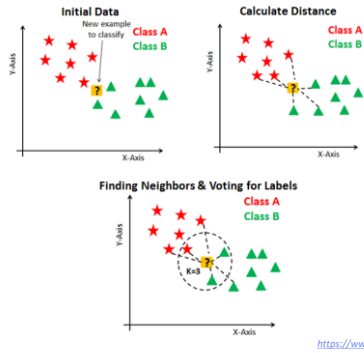
- k-Nearest Neighbor(KNN)
  - is based on feature similarity approach.
    - **It does not have a training phase** to build a model as it is an instance based algorithm
  - is a simple algorithm that can be applied both for classification and regression problems.



https://towardsdatascience.com/building-a-k-nearest-neighbors-k-nn-model-with-scikit-learn-51209555453a

kNN

- k is the number of nearest neighbors.
  - The definition of k is critical to the performance of algorithm
  - Usually if the problem is a binary classification (avoid ties)
    - $k > 1$ and it is odd
    - If $k = 1$ the algorithm is known as the nearest neighbor algorithm.



Goal: Find the k closest instances to the new instance and then classify it based on the majority of votes of its k neighbors.

KNN main steps:

- Calculate distances
- Identify the k closest neighbors
- Find the majority vote based on those k instances

https://www.datacamp.com/community/tutorials/k-nearest-neighbor-classification-scikit-learn

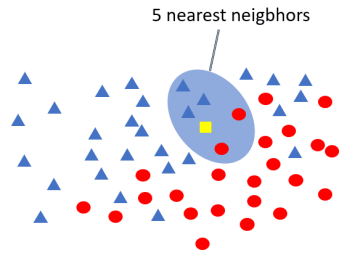
kNN

- KNN algorithm is:
  - non-parametric
    - It does not assume any underlying data distribution, which means that can be applied to all the classification / regression problems (dealing with real world datasets)
  - lazy learning *
    - This is a key aspect in the operation of this algorithm, as it does not generate a model that must be trained, so it does not require any training data.
      - **There is no model**, thus no need for learning or training a model
        - Instance-based learning;
      - **There is no training phase**.

* Lazy Learning is the opposite of Eager Learning , where the prediction of a new instance is performed through a model trained based on a training data set (more demanding in the training phase than in the testing phase)

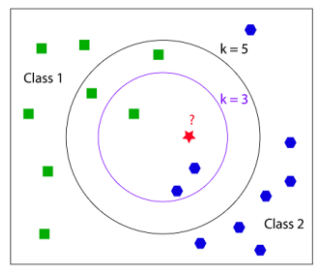
kNN

- KNN algorithm:
  - All the instances are used at the time of prediction, the algorithm simply store the training dataset and waits until classification needs to be performed.
    - All the work is made in the testing phase to classify a new instance.
    - The data is applied only when the algorithm has to make a new prediction which may originate a slow and costly procedure.
      - KNN consider all the instances of the dataset.



kNN

- KNN algorithm is affected by:
  - high dimensionality (the curse of dimensionality)
    - KNN performs better with a lower number of features than a large number of features
  - selection of k:
    - This is an hyper parameters that **has an critical impact** in the performance of KNN



https://machine-learning-tutorial-obi.readthedocs.io/en/latest/content/supervised/knn.html

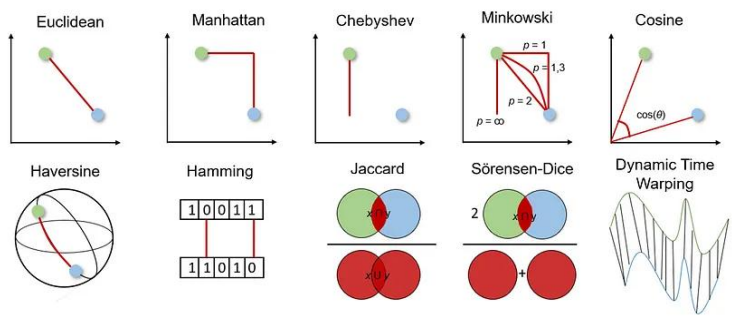
    - There is no optimal number of neighbors (i.e. it depends of the problem complexity, the dataset, number of features, ….)

kNN

- There is no optimal number of neighbors
  - A small number of neighbors **has high variance**, is more susceptible to noise but on the other hand makes the prediction more flexible and faster than a higher number of neighbors.
  - A larger number of neighbors reduces variance but increases computational cost.
  - If the number of classes is even k should be odd.
    - The selection of k usually is an iterative process.
- Scale
  - All the features **should have the same scale**.

kNN

- KNN is based on the identification of the k closest instances.
  - Closest?
    - In order to identify the nearest neighbors a distance function must be applied.
    - Similarly to the selection of k, the choice of the distance function is determinant to the algorithm's performance



https://medium.com/mlearning-ai/a-brief-introduction-to-distance-measures-ac89cbd2298

kNN

- Closest?

Distance Functions between instances kNN

$$\mathbf{u} = [u_1 \dots u_p]^T; \mathbf{v} = [v_1 \dots v_p]^T; \quad p \; number \; of \; features$$

Euclidean	$d(\mathbf{u},\mathbf{v}) = \sqrt{\sum_{i=1}^{p}(u_i - v_i)^2}$
Minkowsky	$d(\mathbf{u},\mathbf{v}) = \sqrt[r]{\sum_{i=1}^{p}\lvert u_i - v_i\rvert^r}$
Manhattan	$d(\mathbf{u},\mathbf{v}) = \sum_{i=1}^{p}\lvert u_i - v_i\rvert$
Chebychev	$d(\mathbf{u},\mathbf{v}) = max_{i=1}^{p}\lvert u_i - v_i\rvert$
Camberra	$d(\mathbf{u},\mathbf{v}) = \sum_{i=1}^{p}\frac{\lvert u_i - v_i\rvert}{\lvert u_i + v_i\rvert}$
…	…

kNN

- KNN Prediction
  - It can be stated as:

$$f\colon R^p \rightarrow C$$

$$c_{\mathbf{x}_q} \leftarrow argmax_{c \in C}\sum_{i=1}^{k}\omega_i \delta(c, c_{\mathbf{x}_i}) \qquad \delta(a,b) = \begin{cases} 1 & if \quad a = b \\ 0 & if \quad a \neq b \end{cases}$$

where:

$\mathbf{x}_q$ is the query instance with p features

$\mathbf{x}_1, \dots, \mathbf{x}_k$ are the k nearest neighbors of the query instance

ω_i is the weight factor:

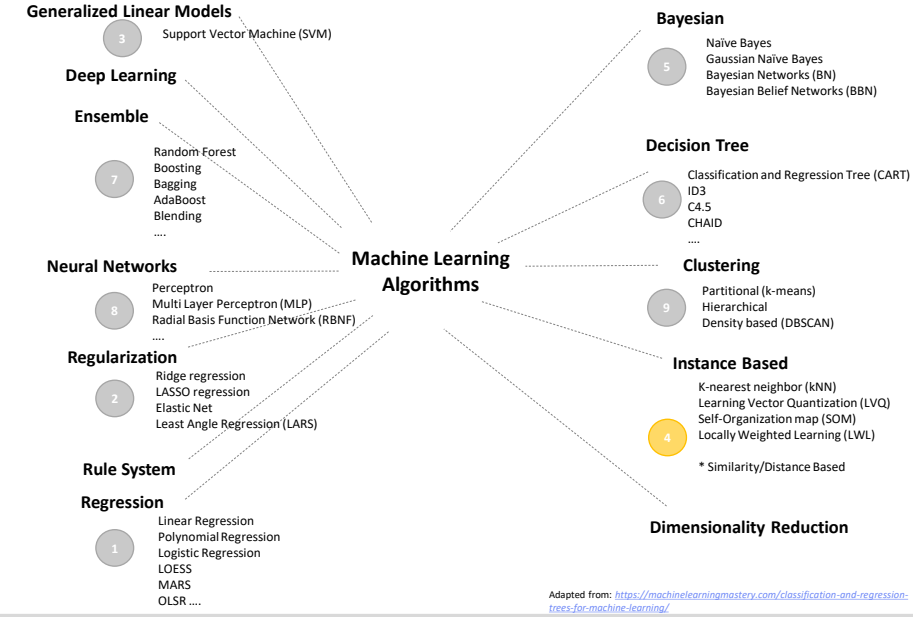
$\omega_i = 1$ if all the votes have the same importance

ω_i can also be defined according to the distance to the several neighbors (increasing proportionally to the reduction of the distance between a given neighbor and the query instance $\mathbf{x}_q$)

Distances/Similarity

Supervised Classification

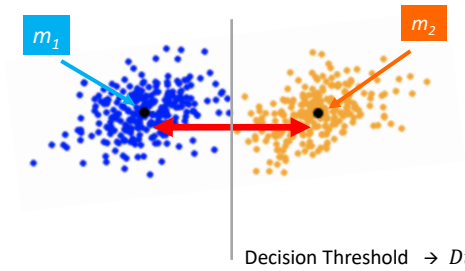
Machine Learning Algorithms



Adapted from: https://machinelearningmastery.com/classification-and-regression-trees-for-machine-learning/

Distances / Similarity

- **Closely related** with the concept **of groups of instances** (clusters) based on similarity/dissimilarity
  - Important: Supervised Classification Approach



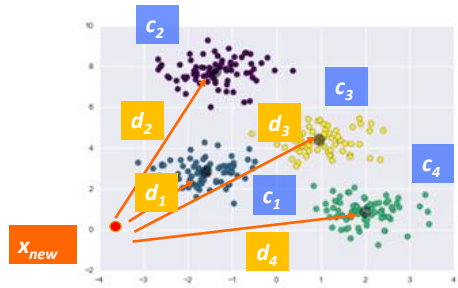
Decision Threshold → $Dt = \frac{m1+m2}{2}$

  - A new instance belongs to the closest center:

$$\begin{cases} if \;\; \mathbf{x}_i < Dt \;\; then \;\; class\; 1 \\ if \;\; \mathbf{x}_i \geq Dt \;\; then \;\; class\; 2 \end{cases}$$

Distances / Similarity

- Similarity
  - Calculated based on each dimension $\mathbf{X} = [X_1, X_2, \ldots, X_p]$
  - It is applied considering simultaneously all the features X_i
    - 1. For each class calculate the respective center considering all the $X_i \;\; i = 1, \ldots, p$
    - *2.* A new instance $\mathbf{x}_{new}$ belongs to the closest center



$\mathbf{x}_{new}$?

$d_1 < d_2 < d_3 < d_4$

$\mathbf{x_{new}} \rightarrow \boldsymbol{c_1}$

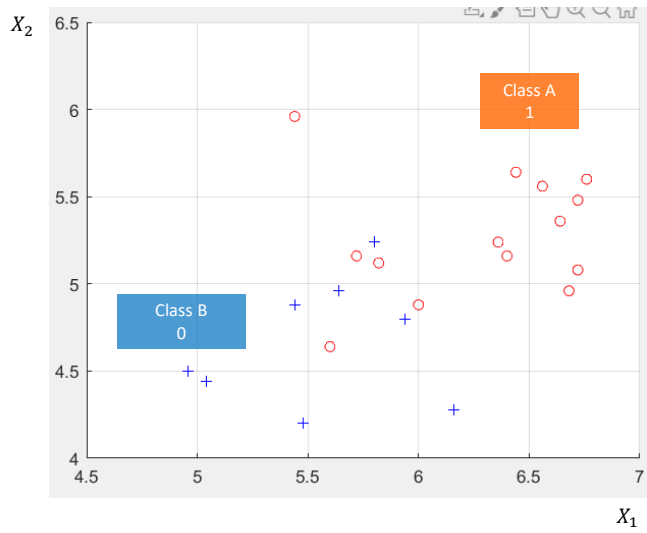
Distances / Similarity

- Example: **Classification** based on Similarity / **Supervised** Learning

Classe A		Classe B	
X_1	X_2	X_1	X_2
6.36	5.24	6.16	4.28
5.82	5.12	5.80	5.24
5.72	5.16	5.64	4.96
6.44	5.64	5.44	4.88
6.40	5.16	5.04	4.44
6.56	5.56	4.96	4.50
6.64	5.36	5.48	4.20
6.68	4.96	5.94	4.80
6.72	5.48		
6.76	5.60		
6.72	5.08		
6.00	4.88		
5.60	4.64		
5.44	5.96		
$N_A = 14$		$N_B = 8$	

Adapted from:
Jorge Henriques (University of Coimbra)

Distances / Similarity

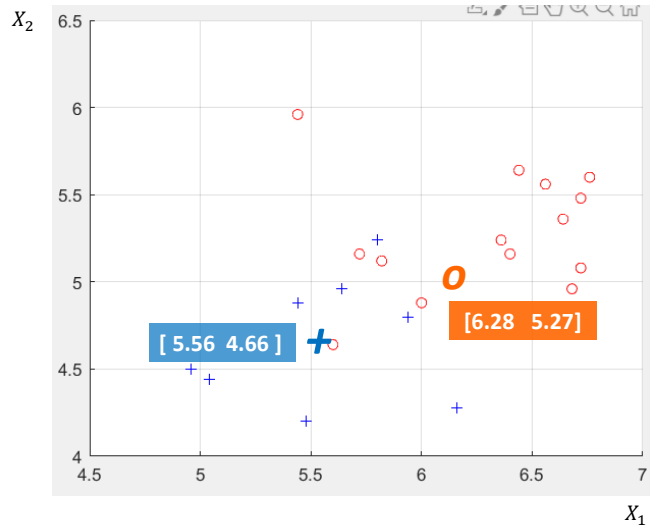


Distances / Similarity

Ex. Class B

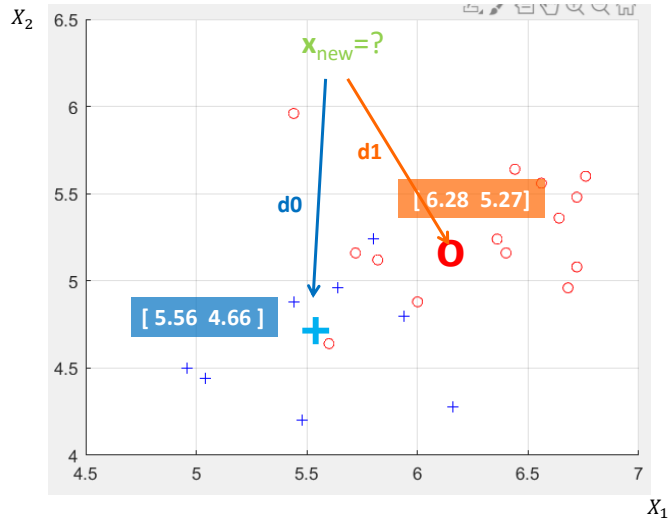
6.16	4.28
5.80	5.24
5.64	4.96
5.44	4.88
5.04	4.44
4.96	4.50
5.48	4.20
5.94	4.80
5.56	***4.66***

mean values



Distances / Similarity

Classify ?



Distances / Similarity

Classe 1 – 14 elementos
TP = 12
FN = 2
Classe 0= 8 elementos
TN = 7
FP = 1

	$C=0$	$C=1$
$\hat{C}=0$	TN = 7	FN=2
$\hat{C}=1$	FP=1	TP=12

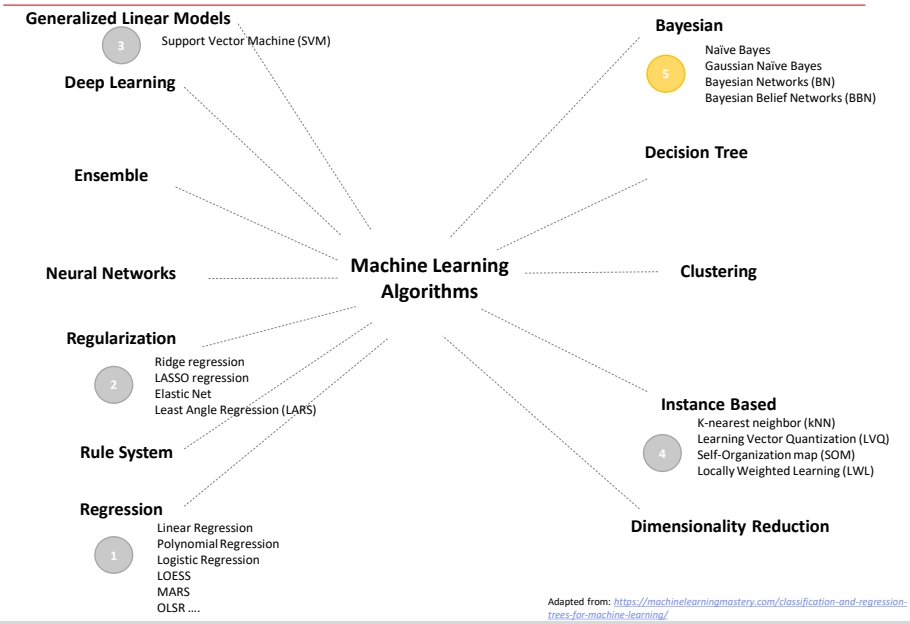
$$SE = \frac{TP}{TP+FN} = \frac{12}{14} = 0.86$$
$$SP = \frac{TN}{TN+FP} = \frac{7}{8} = 0.88$$



Bayesian Classifiers - Naïve Bayes

Classification

Machine Learning Algorithms



Adapted from: https://machinelearningmastery.com/classification-and-regression-trees-for-machine-learning/

Naïve Bayes

- Probabilities:
  - The marginal Probability
    - is the probability of an event A of one random variable X regardless the outcomes of other random variables, e.g. $P(A)$.
      - Example: The probability that a drawn card is red
        - $P(red) = 0.5$
  - The joint probability
    - is the probability of two (or more) simultaneous events A, B from two random variables X *and* Y .
    - Usually is referred as the probability of A *and* B and denoted as $P(A, B)$
      - Example: The probability that a drawn card is a four and it is red
        - $P(four, red) = \frac{1}{13} \times \frac{1}{2} = \frac{1}{26}$

Naïve Bayes

- Probabilities:
  - The conditional probability
    - is the probability of one event A given the occurrence of another event B from two random variables X and Y.
    - It is referred as the probability of A given B and it is denoted by $P(A|B)$
      - Example: The probability, given that the drawn card is red, that it is a four.
        - $P(four|red) = \frac{2}{26} = \frac{1}{13}$

Naïve Bayes

- The joint probability is symmetrical:

$$P(A, B) = P(B, A)$$

- The conditional probability is not symmetrical:

$$P(A|B) \neq P(B|A)$$

- The joint probability relates with conditional probability through the product rule:

$$P(A, B) = P(A|B) * P(B)$$

$$P(B, A) = P(B|A) * P(A)$$

- Events A and B are independent if:

$$P(A, B) = P(A).P(B)$$

Naïve Bayes

- Bayesian classifiers are probabilistic classifiers
  - These classifiers implement particular structures of Bayesian networks:
    - Their goal is to assign a class label to instances described by a given set of attributes.
    - In a classification process, a probabilistic learning algorithm outputs a probability that a given instance belongs to each class.
  - The classification relies of the Bayes Rule:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

  - The class of C with the highest probability given the value of attribute X is the final prediction

Naïve Bayes

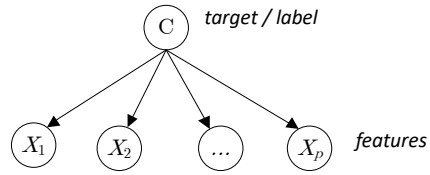
- Bayes Rule:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

  - X is an observation / feature (e.g. Clinical Exam)
  - C a hypothesis (e.g., have a disease / not have a disease)
  - $P(C|X)$:
    - Posterior probability, the probability of the hypothesis C after having seen the observation X
      - e.g., Probability to have a disease given the results of a clinical exam.
  - $P(C)$
    - Prior belief, the probability of the hypothesis before seeing any observation
      - e.g. prevalence of the disease.
  - $P(X\,|C)$
    - Likelihood, the probability of the observation if the hypothesis is true
      - Sensitivity of the clinical exam.

Naïve Bayes

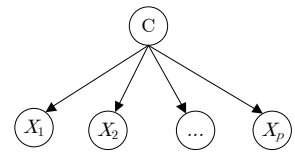
- It assumes a particular configuration of a Bayesian Network
  - A directed acyclic graph (DAG)
    - Only one parent (unobserved node/ **target**)
    - Several children (observed nodes / **features**).



    - Classification relies on Bayes rule to predict the class of C **with the highest probability given a set** of attributes $\{X_1, \ldots, X_p\}$

Naïve Bayes

- Despite its simple configuration, naïve Bayes is:
  - Computationally efficient
  - **Can deal with incomplete information**
  - Presents a predictive performance competitive with other classifiers



  - usually it adopts the following notation
    - $\mathbf{X} = [X_1 \ldots X_p]$ is a vector of random variables that contains the observed attributes;
    - $\mathbf{x} = [x_1 \ldots x_p]$ a particular instance that contains the observed values of the different p attributes;
    - C is a random variable that denotes the class of an instance
    - c is a particular class label.

Naïve Bayes

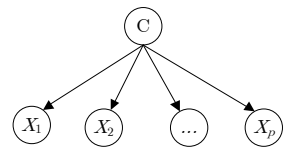
- Its structure imposes a strong independence condition:
  - all the attributes X_i **are conditionally independent** given the value of the class C.
    - A is independent of B given C whenever $P(A|B,C) = P(A|C)$ for all possible values of A, B, C with $P(C) > 0$
    - As all the X_i , $i = 1 \ldots . p$ are independent, $P(\mathbf{X})$ is given by:

$$P(\mathbf{X}) = \prod_{i=1}^{p} P(X_i)$$

  - Frequently the conditional independence between attributes is unrealistic
    - However even if this condition is not verified, naïve Bayes often presents a performance comparable to other classifiers.

Naïve Bayes

- So, considering a graph structure G comprising:
  - The class variable C that is the root node
  - Set of discrete random variables (attributes) $\{X_1, \ldots, X_p\}$ where each attribute has the class variable as its unique parent.



$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

    - The conditional probability $P(C|\mathbf{X})$ is given by:

$$P(C|\mathbf{X}) = P(C|X_1, \ldots, X_p) = \alpha P(C) \prod_{i=1}^{p} P(X_i|C)$$

where α is a normalization constant

Naïve Bayes

- Prediction
  - The final classification is given by:

$$f\colon R^p \rightarrow C$$

$$c_{\mathbf{x}_q} \leftarrow argmax_{c_j \in C}\ (\alpha P(c_j) \prod_{i=1}^{p} P(x_i|c_j)$$

where:

$\mathbf{x}_q$ is the query instance with p features

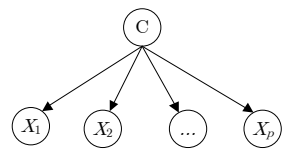
c_j is a mutually exclusive class of C

x_i is the value of attribute X_i that belongs to the query instance $\mathbf{x}_q = [x_1 \ldots x_p]$

$$\alpha = \frac{1}{P(\mathbf{X}=\mathbf{x}_q)}$$

Naïve Bayes

- The structure of naïve Bayes classifier is completely defined:



- Therefore the **learning process is related only with the direct calculation of model parameters**.
  - The model has to learn from the training data set, the **conditional probability** $P(X_i|C)$ of each attribute X_i given the class C as well as the **prior probability** $P(C)$.
  - The discretization of numeric attributes has a key impact:
    - In the construction of the conditional probability table $P(X_i|C)$ and therefore in the performance of the classifier.

Naïve Bayes

- Learning Process
  - Conditional Probabilities
    - An attribute X_i can be either:
      - Categorical
      - Numeric.
    - The different nature of the attributes **has a critical importance** in the probabilities estimation

Naïve Bayes

- Learning Process
  - Numerical data:
    - impose an additional difficulty, as the attributes X_i may assume a large or infinite number of values, thus the probability of a given value $X_i = x_k$ can be infinitely small.
    - Two possible solutions:
      - Estimate probabilities directly from the density function that gives the distribution of X_i over a specific class c_j.
        - e.g. Normal distribution $f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$ $(\mu - mean;\ \sigma - std)$
        - Frequently the distribution is unknown!
      - Discretization

Naïve Bayes

- Learning Process
  - Discretization
    - A categorical attribute X_i^* is formed from X_i, where each value of X_i^*=x_k^* corresponds to an interval $]a_1, a_2]$ of X_i
    - Each $x_k \in]a_1, a_2]$ is replaced by the corresponding x_k^*
    - The determination of Conditional Probabilities is performed:

$$P(X_i = x_k^* | C = c_j) = \frac{\sum_{z=1}^{m}(X_i = x_k^* \wedge C = c_j)}{\sum_{z=1}^{m}(C = c_j)}$$

where:

$c_j \in C = \{c_1, \ldots, c_l\}$ that represent l mutually exclusive classes

x_k^* denotes a particular value of feature/attribute X_i

m is the total number of instances (all the instances of the training dataset)

Naïve Bayes

- Learning Process
  - Discretization Methods
    - To build the conditional probability tables the discretization of variables must be properly performed.
      - There are several methods to perform that operation:
        - Equal Width Discretization (EWD);
        - Equal Frequency Discretization;
        - Fuzzy Discretization,
        - Entropy Minimization Discretization
        - Iterative Discretization
        - Proportional k-interval Discretization
        - Weighted Proportional k-interval Discretization
        - …

Naïve Bayes

- Discretization Methods
  - Equal Width Discretization (EWD)
    - Equal width discretization creates ρ intervals of equal width.
    - Each interval has width Ω given through:

$$\Omega = \frac{max(x_i) - min(x_i)}{\rho}$$

  - Equal Frequency Discretization
    - Equal frequency discretization divides the sorted values into ρ intervals so that each interval contains identical number of instances $\frac{m}{\rho}$ where m is the total number of instances.
  - Both methods are frequently used for naïve Bayes classifiers due to their simplicity and reasonable good performance.

Naïve Bayes

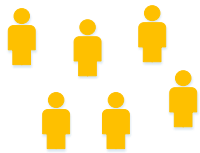
- Categorical/Ordinal data:
  - Usually assumes a small number of possible values, which allows a reliable estimation of probabilities:
    - $P(C = c_j)$ that can be estimated from the frequency of instances with $C = c_j$
    - $P(X_i = x_k | C = c_j)$ that can be estimated from the frequency of instances $X_i = x_k \wedge C = c_j$ divided by the total of instances $C = c_j$

$$P(X_i = x_k | C = c_j) = \frac{\sum_{z=1}^{m}(X_i = x_k \wedge C = c_j)}{\sum_{z=1}^{m}(C = c_j)}$$

Naïve Bayes

Training Data Set

$Death = 6$
$P(D) = 0.33$



$x_1 =$ diabetes

5

$P(x_1|D) = \frac{5}{6} = 0.83$

$P(\bar{x}_1|D) = \frac{1}{6} = 0.17$

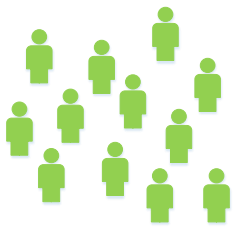
$x_2 =$ high colesterol

4

$P(x_2|D) = \frac{4}{6} = 0.67$

$P(\bar{x}_2|D) = \frac{2}{6} = 0.33$

$Survival = 12$
$P(\bar{D}) = 0.67$



3

$P(x_1|\bar{D}) = \frac{3}{12} = 0.25$

$P(\bar{x}_1|\bar{D}) = \frac{9}{12} = 0.75$

1

$P(x_2|\bar{D}) = \frac{1}{12} = 0.08$

$P(\bar{x}_2|\bar{D}) = \frac{11}{12} = 0.92$

Naïve Bayes

Training Data Set

$P(D) = 0.33$

$x_1 =$ diabetes

$P(x_1|D) = 0.83$

$P(\bar{x}_1|D) = 0.17$

$x_2 =$ high cholesterol

$P(x_2|D) = 0.67$

$P(\bar{x}_2|D) = 0.33$

$P(\bar{D}) = 0.67$

$P(x_1|\bar{D}) = 0.25$

$P(\bar{x}_1|\bar{D}) = 0.75$

$P(x_2|\bar{D}) = 0.08$

$P(\bar{x}_2|\bar{D}) = 0.92$

New instance:

Diabetes
No high cholesterol

P(C|X) =?

$C \rightarrow \{D(1), \bar{D}(0)\}$

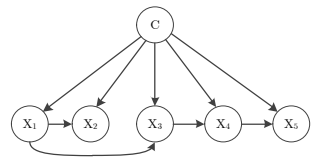
$P(D|X) = P(D) \times P(x_1|D) \times P(\bar{x}_2|D) = 0.33 \times 0.83 \times 0.33 = 0.091$

$P(\bar{D}|X) = P(\bar{D}) \times P(x_1|\bar{D}) \times P(\bar{x}_2|\bar{D}) = 0.67 \times 0.25 \times 0.92 = 0.154$ ←

Survival

Naïve Bayes

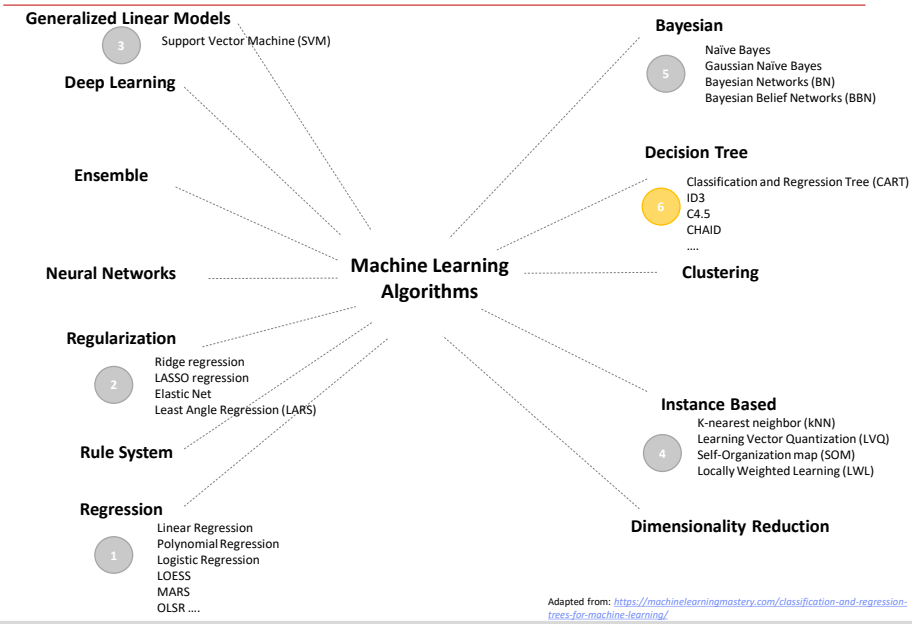
- Semi Naïve Bayes Methods
  - Naive Bayes makes the assumption that all the attributes X_i are conditionally independent given the value of class C .
    - Although in many current situations this assumption may be unrealistic.
    - Some methods designated as semi naïve Bayesian attenuate this attribute interdependence problem
      - There are two main approaches to circumvent the “independence assumption”:
        - Methods that define a new set of attributes;
          - Backward Sequential Elimination (BSE), Forward Sequential Selection (FSS), …
        - Methods that define new interdependencies between attributes.
          - Tree-Augmented Naïve Bayes (TAN), Super-parent TAN (SP-TAN), NBTree, …



Decision Trees

Classification

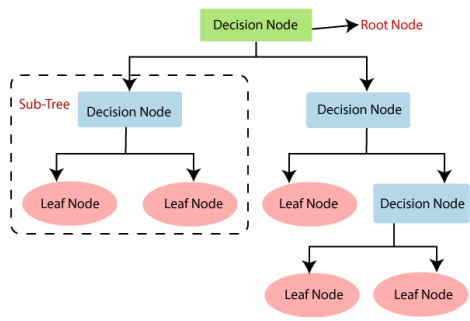
Machine Learning Algorithms



Adapted from: https://machinelearningmastery.com/classification-and-regression-trees-for-machine-learning/

Decision Tree

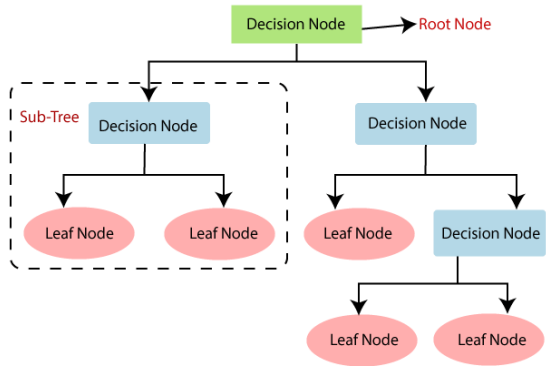
- Decision Trees can perform both classification and regression tasks
  - They are also the basis of the Random Forest Classifiers



https://www.javatpoint.com/machine-learning-decision-tree-classification-algorithm

- **Root Node** is where from the decision tree starts. It represents the entire dataset, which is divided into two sets in order to allow further decisions.
- **Leaf Nodes** are the final output node (no child node, no further splitting). They contain the output class.
- **Parent/Child node** based on the hierarchical structure of the tree.

Decision Tree

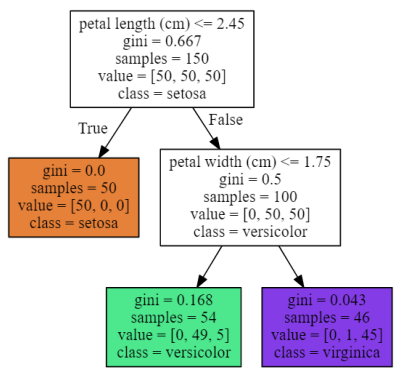


https://www.javatpoint.com/machine-learning-decision-tree-classification-algorithm

- **Splitting** divides the decision node/root node into sub-nodes according to the given conditions.
- **Branch/Sub Tree** is a tree created by splitting the original tree.
- **Pruning** is the process of removing the unwanted branches from the tree.

Decision Tree

- A decision tree plot in scikit-learn provides some data about the respective creation:



Node Attributes

gini: measures the impurity of the node, a node is pure (gini=0) if all the training instances belong to the same class

samples: number of samples that it applies for

value: how many training instances of each class this node applies to

class: if **it is a leaf node** (does not have any child nodes) it refers to the predicted class

Decision Tree

- Attribute Selection Measures (ASM)
  - How to select the best attribute for the root node and for sub-nodes?
    - Two main alternatives:

Gini Index *Entropy*

- Gini's Impurity tends to compute faster, while entropy tends to produce slightly more balanced trees

Decision Tree

- Gini Index
  - is an impurity measure given by:

$$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$$

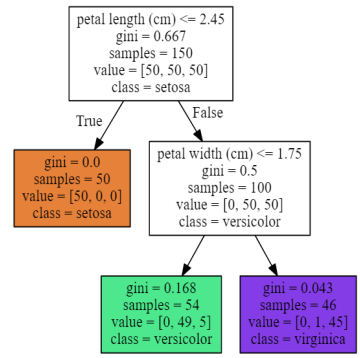
where $p_{i,k}$ is the ratio of k instances among the training instances in the ith node considering the n classes

- The Gini index should be as low as possible.
- The CART (Classification and Regression Tree) algorithm uses the Gini index to create binary splits.

Decision Tree

- Gini Index
  - $G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$

$$G_i = 1 - \left(\frac{50}{50}\right)^2 - (\frac{0}{50})^2\text{-}(\frac{0}{50})^2 = 0$$



$$G_i = 1 - \left(\frac{0}{54}\right)^2 - (\frac{49}{54})^2\text{-}(\frac{5}{54})^2 \approx 0.168$$

Decision Tree

- Example: Gini Index ($G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$)
  - Decision tree to classify whether a client is going to buy a product (yes/no) based on Age (young/old) and Income (low/high).

#	Age	Income	Buy
1	Young	Low	No
2	Young	Low	No
3	Young	High	Yes
4	Young	High	No
5	Old	Low	Yes
6	Old	Low	Yes
7	Old	High	Yes
8	Old	High	Yes

  - Gini Index of the parent node:

$$p_{Yes} = {}^{5}/_{8} = 0.625,\ p_{No} = {}^{3}/_{8} = 0.375$$

$$G(parent) = 1 - (0.625^2 + 0.375^2) = 0.468$$

Decision Tree

- Gini Index ($G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$)
  - Age (Candidate Split):
    - Age="Young" (4 instances) Yes=1; No=3
      - $p_{Yes} = {}^1/_4 = 0.25$, $p_{No} = {}^3/_4 = 0.75$
      - $G(young) = 1 - (0.25^2 + 0.75^2) = 0.375$
    - Age="Old" (4 instances) Yes=4; No=0
      - $p_{Yes} = {}^4/_4 = 1$
      - $G(Old) = 1 - (1^2 + 0) = 0$
    - Gini After Split Age (each group 4 instances; $weight = 4/8 = 0.5$):
      - $G_{split,Age} = 0.5 \times 0.375 + 0.5 \times 0 = 0.1875$
    - Gini Reduction (Age):
      - $\Delta G_{Age} = 0.468 - 0.187 = 0.281$

#	Age	Income	Buy
1	Young	Low	No
2	Young	Low	No
3	Young	High	Yes
4	Young	High	No
5	Old	Low	Yes
6	Old	Low	Yes
7	Old	High	Yes
8	Old	High	Yes

Decision Tree

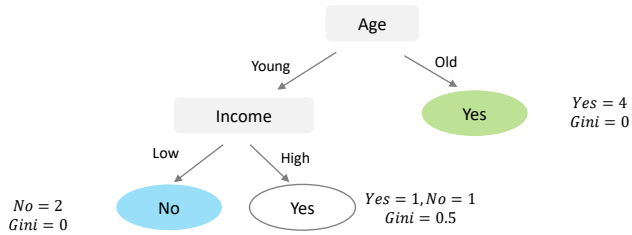
- Gini Index ($G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$)
  - Income (Candidate Split):
    - Income="Low" (4 instances) Yes=2; No=2
      - $p_{Yes} = {}^2/_4 = 0.5$, $p_{No} = {}^2/_4 = 0.5$
      - $G(Low) = 1 - (0.5^2 + 0.5^2) = 0.5$
    - Income="High" (4 instances) Yes=3; No=1
      - $p_{Yes} = {}^3/_4 = 0.75$, $p_{No} = {}^1/_4 = 0.25$
      - $G(high) = 1 - (0.75^2 + 0.25^2) = 0.375$
    - Gini After Split on Income (each group 4 instances; $weight = 4/8 = 0.5$):
      - $G_{split,Income} = 0.5 \times 0.5 + 0.5 \times 0.375 = 0.4375$
    - Gini Reduction (Income):
      - $\Delta G_{Income} = 0.468 - 0.4375 = 0.03125$

#	Age	Income	Buy
1	Young	Low	No
2	Young	Low	No
3	Young	High	Yes
4	Young	High	No
5	Old	Low	Yes
6	Old	Low	Yes
7	Old	High	Yes
8	Old	High	Yes

Decision Tree

- Gini Index ($G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$)
  - Gini Reduction (Age):
    - $\Delta G_{Age} = 0.468 - 0.187 = 0.281$
  - Gini Reduction (Income):
    - $\Delta G_{Income} = 0.468 - 0.4375 = 0.03125$
  - Age reduces impurity (Gini Index) much more than Income
    - Age is chosen as the root (best split).

#	Age	Income	Buy
1	Young	Low	No
2	Young	Low	No
3	Young	High	Yes
4	Young	High	No
5	Old	Low	Yes
6	Old	Low	Yes
7	Old	High	Yes
8	Old	High	Yes



Decision Tree

- Entropy
  - It is a measure of disorder or impurity in a node and ranges from 0 to 1.
  - **Entropy should be as low as possible**
    - 0 means a leaf node with all the instance belonging to the same class

$$H_i = -\sum_{k=1}^{n} p_{i,k} \log_2(p_{i,k})$$

$p_{i,k}$: ratio of k instances among the training instances in the ith node considering the n classes

- A decision tree algorithm always tries to **minimize entropy (maximize the information gain IG)**, so for the ith node:

$$\mathrm{IG}(H_{parent}; H_{children}) = H_{parent} - H_{children}$$

The higher the information gain, the better the split!

Decision Tree

- Example: Entropy ($H_i = -\sum_{k=1}^{n} p_{i,k} \log_2(p_{i,k})$)
  - Decision tree to classify whether a client is going to buy a product (yes/no) based on Age (young/old) and Income (low/high).

#	Age	Income	Buy
1	Young	Low	No
2	Young	Low	No
3	Young	High	Yes
4	Young	High	No
5	Old	Low	Yes
6	Old	Low	Yes
7	Old	High	Yes
8	Old	High	Yes

  - Entropy of the parent node:

$$p_{Yes} = {}^{5}/_{8} = 0.625,\ p_{No} = {}^{3}/_{8} = 0.375$$

Entropy is given in **bits,** a **base-2 logarithm**, that measures information in terms of **binary decisions**.
One bit represents the amount of uncertainty removed by answering a single **yes/no** question.

$$H_{parent} = 1 - (0.625 \log_2 0.625 + 0.375 \log_2 0.375) = 0.9544340029\ bits$$

Decision Tree

- Entropy ($H_i = -\sum_{k=1}^{n} p_{i,k} \log_2(p_{i,k})$)
  - Age (Candidate Split):
    - Age="Young" (4 instances) Yes=1; No=3
      - $p_{Yes} = {}^{1}/_{4} = 0.25,\ p_{No} = {}^{3}/_{4} = 0.75$
      - $H_{young} = -[0.25 \log_2 0.25 + 0.75 \log_2 0.75] = 0.8112781245\ bits$
    - Age="Old" (4 instances) Yes=4; No=0
      - $p_{Yes} = {}^{4}/_{4} = 1$
      - $H_{old} = 0$ (Zero entropy means no uncertainty → 0 bits to know the class)
    - Entropy After Split Age (each group 4 instances; $weight = 4/8 = 0.5$):
      - $H_{split,Age} = 0.5 \times 0.8112781245 + 0.5 \times 0 = 0.4056390622\ bits$
    - Information Gain (Age):
      - $IG_{Age} = H_{parent} - H_{split,Age} = 0.9544340029 - 0.4056390622 = 0.54879407\ bits$

#	Age	Income	Buy
1	Young	Low	No
2	Young	Low	No
3	Young	High	Yes
4	Young	High	No
5	Old	Low	Yes
6	Old	Low	Yes
7	Old	High	Yes
8	Old	High	Yes

Decision Tree

- Entropy ($H_i = -\sum_{k=1}^{n} p_{i,k} \log_2(p_{i,k})$)
  - Income (Candidate Split):
    - Income="Low" (4 instances) Yes=2; No=2
      - $p_{Yes} = {}^2/_4 = 0.5$, $p_{No} = {}^2/_4 = 0.5$
      - $H_{low} = -[0.5 \log_2 0.5 + 0.5 \log_2 0.5] = 1\ bits$
    - Income="High" (4 instances) Yes=3; No=1
      - $p_{Yes} = {}^3/_4 = 0.75$, $p_{No} = {}^1/_4 = 0.25$
      - $H_{high} = -[0.75 \log_2 0.75 + 0.25 \log_2 0.25] = 0.8112781245\ bits$
    - Entropy After Split Income (each group 4 instances; $weight = 4/8 = 0.5$):
      - $H_{split,Income} = 0.5 \times 1 + 0.5 \times 0.8112781245 = 0.9056390622\ bits$
    - Information Gain (Income):

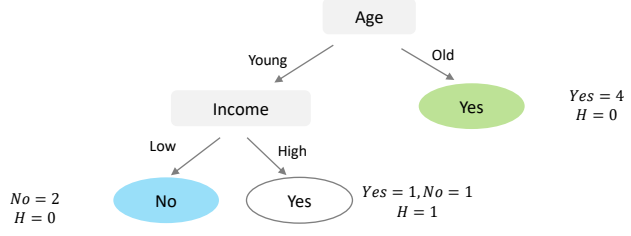
      $IG_{Income} = H_{parent} - H_{split,Income} = 0.9544340029 - 0.9056390622 = 0.0487949407\ bits$

#	Age	Income	Buy
1	Young	Low	No
2	Young	Low	No
3	Young	High	Yes
4	Young	High	No
5	Old	Low	Yes
6	Old	Low	Yes
7	Old	High	Yes
8	Old	High	Yes

Decision Tree

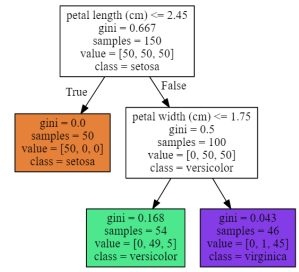
- Entropy ($H_i = -\sum_{k=1}^{n} p_{i,k} \log_2(p_{i,k})$)
  - IG (Age):
    - $IG_{Age} = 0.54879\ bits$
  - IG (Income):
    - $IG_{Income} = 0.04879\ bits$
  - Age reduces uncertainty much more than Income
    - Age is chosen as the root (best split).

#	Age	Income	Buy
1	Young	Low	No
2	Young	Low	No
3	Young	High	Yes
4	Young	High	No
5	Old	Low	Yes
6	Old	Low	Yes
7	Old	High	Yes
8	Old	High	Yes



Decision Tree

- Training
  - CART (Classification and Regression Tree) algorithm



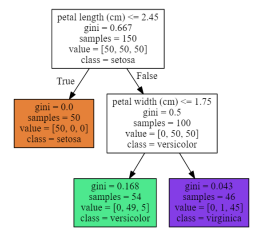
  - **Step1:**
    - Split the training into two subsets using a single feature k and a threshold t_k.
    - The pair (k, t_k) is selected considering that it produces the purest subsets, according the minimization of the cost function:

$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

      - where $G_{left/right}$ measures the impurity of the left/right subset and $m_{left/right}$ is the number of instances in the left/right subset, m is the number of instances to be split.

Decision Tree

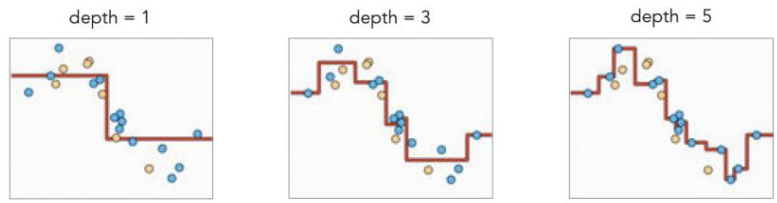
- Training
  - CART (Classification and Regression Tree) algorithm



  - **Step 2:** After this initial split, the same procedure is applied to both subsets and so on to the subsequent subsets (recursively).
    - The process continues until it reaches a stop condition:
      - Cannot find a split that reduces impurity
      - Maximum depth (defined by the hyper parameter max_depth)

Decision Tree

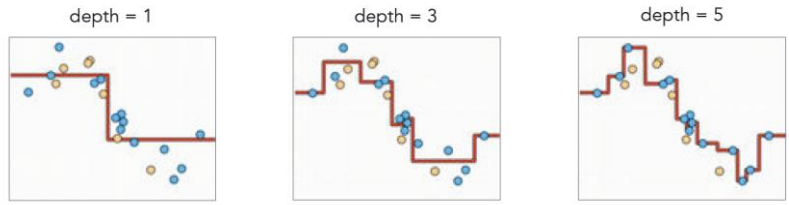
- Regularization hyper parameters:
  - Decision trees is a non parametric model, i.e. the number of parameters is not defined prior to the training process (unlike e.g. linear models)
    - Then, the model structure is prone to stick closely to data (high risk of overfitting).
      - Actually, if left unconstrained the tree structure will adapt very closely to training data, probably creating an overfitting situation.



Jeremy Watt, Machine Learning Refined 2nd edition, ISBN: 1108480721, Cambridge University Press,2020

Decision Tree

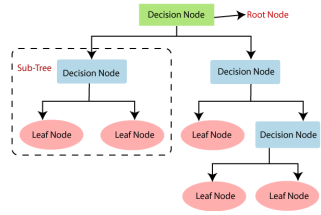
- Regularization hyper parameters:
  - **To avoid this situation**, the freedom degrees of Decision Tree must be controlled through the training phase (regularization).
    - One of the most applied hyperparameters to regularize the model is the **max-depth**, which defines the maximum depth of the DT (pre-pruning process).



Jeremy Watt, Machine Learning Refined 2nd edition, ISBN: 1108480721, Cambridge University Press,2020

Decision Tree

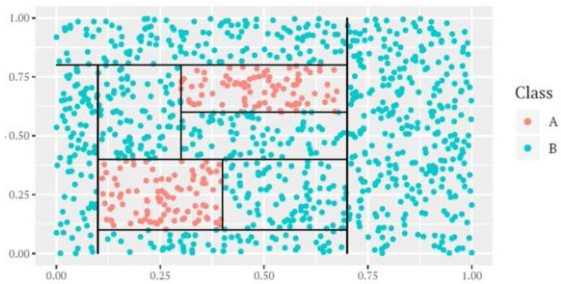
- Regularization hyper parameters:
  - There are other parameters to constrain the structure of DT:
    - **max_leaf_nodes**: maximum number of leaf nodes

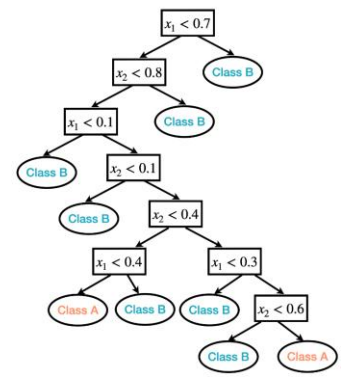


    - **min_samples_split**: minimum number of samples that a node must have to allow a further split
    - **min_sample_leaf**: minimum number of samples that a leaf node must have
    - **min_weight_fraction_leaf**: same as min_sample_leaf but expressed as a fraction of the total number of instances
    - **max_features**: maximum number of features that are evaluated for each splitting

Decision Tree

- DT are prone to Overfitting

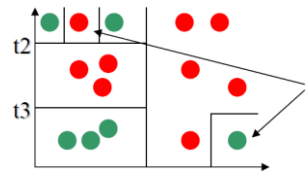




https://bookdown.org/hailiangdu/Lecture_notes/tree-based-models.html

Decision Tree

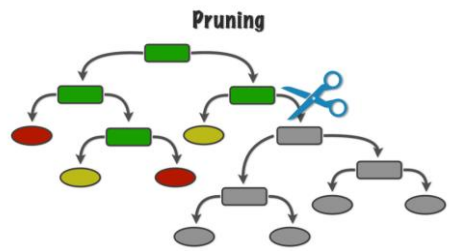
- Regularization
  - Post-pruning
    - If a tree has been built without early stopping (pre-pruning) settings, probably there will be an overfitting situation.
      - CART algorithm repeatedly partitions data into smaller and smaller subsets until those final subsets are homogeneous in terms of the outcome variable.
        - The tree has learned the data exactly **but has no generalization capability**.
          - Useless!



Possibly just noise, but the tree is grown larger to capture these examples

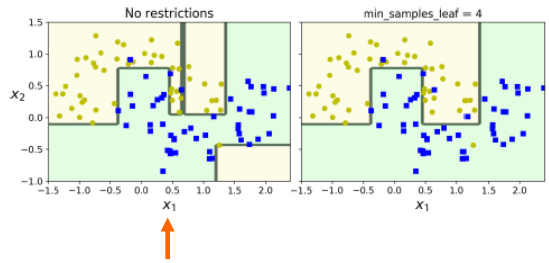
Decision Tree

- Regularization
  - Pruning (post-pruning) considers an initially excessively complicated tree and cutting off its leaves, cutting off redundant leaves and branches.
    - It is a tree-specific form of regularization.
    - There are several methods/algorithms for pruning
      - e.g. Error-Based Pruning (algorithm C 4.5)
        - https://ieeexplore.ieee.org/document/589207



Decision Tree

- Remember!
  - Decision trees are very prone to overfitting
    - The definition of **some of the previous hyper parameters/pruning processes** is critical to avoid/minimize the risk of overfitting and keep the generalization capability of the classifier

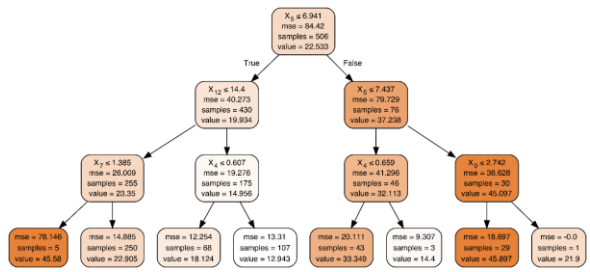


Overfitting Situation

Geron A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems 3rd Edition, ISBN-13: 978-1492032649, O'Reilly 2022

Decision Tree

- Regression
  - Instead of predicting a class, a value is predicted in each node



https://explained.ai/decision-tree-viz/images/boston-scikit.png

  - The prediction value **is the average target value** of the training instances assigned to that leaf node.
    - Each leaf node has a value for MSE that the algorithm seeks to minimize
    - The algorithm splits the instances in a way that makes most training instances as close as possible to the predicted value.

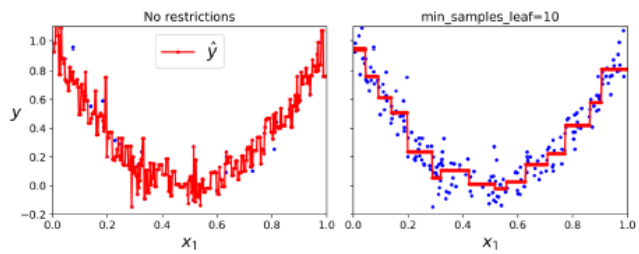
Decision Tree

- Regression
  - CART Algorithm
    - It is very similar to the classification situation, but instead to minimize impurity, here MSE is applied

$$J(k, t_k) = \frac{m_{left}}{m} MSE_{left} + \frac{m_{right}}{m} \text{MSE}_{right} \qquad \text{where} \begin{cases} MSE_{node} = \sum_{i \in node} (\hat{y}_{node} - y^{(i)})^2 \\ \hat{y}_{node} = \frac{1}{m_{node}} \sum_{i \in node} y^{(i)} \end{cases}$$

Decision Tree

- Regression
  - Similarly to classification, the Decision Tree Regressor is also susceptible to overfitting, so some regularization has to be implemented.



Overfitting Situation